

Coursework Breakdown

1. Search for a C Programming Code for a File Encryption Algorithm

- Start by researching a C-based file encryption algorithm like AES, DES, or any simpler ones like XOR encryption.
- A good starting point might be GitHub or other open-source platforms where encryption algorithms are available.
- You can search for "C program file encryption" or look at well-known encryption libraries in C, like OpenSSL, for inspiration or to directly use their implementation.

2. Implement the Code

- Once you've chosen an encryption algorithm, carefully study the provided code, ensuring you understand how it works.
- Implement the code in your local environment (Minix, Linux, or any other Unix-based system).
- If you're using a Linux OS, open a terminal and create a new file for the code (for example, `encryption.c`) and paste the implementation there.

3. Test the Code

- Compile and run the program using a C compiler (`gcc` or `clang` on Unix systems).
- Test the encryption functionality by inputting files and seeing if the encryption/decryption process works correctly.
- Example compilation command:

```
bash
```

 Copy code

```
gcc encryption.c -o encryption
./encryption input.txt output.txt
```

4. Install the Header Files

- Install any necessary header files to `/usr/include` . This is for libraries you might need that aren't natively available.
- Use the following command to move the header files to the right directory:

bash

 Copy code

```
sudo mv "NAME_OF_HEADER_FILE" /usr/include
```

- Make sure you use the actual name of the header file from your encryption library or project.

5. Install the Software

- After you've compiled the code and made sure it works, you can move the executable file to `/usr/bin/` so that it's available system-wide.
- Use the following command to install the software:

bash

 Copy code

```
sudo mv "NAME_OF_EXECUTABLE_FILE" /usr/bin/
```

- This step makes sure that the executable can be run from anywhere in the system.

6. Verify the Software

- After installation, open a terminal and try running the software from any directory to ensure it is correctly installed and can be executed from the system-wide path.
- Run the command:

bash

 Copy code

```
encryption input.txt output.txt
```

If the program works as expected, the encryption should be applied, and you will know the software is installed correctly.

Key Points About Header Files:

1. File Extension:

- Header files typically have a `.h` extension (e.g., `stdio.h`, `stdlib.h`, `math.h`).

2. Content of Header Files:

- Function Declarations:** They declare the functions used in your program but don't provide the actual implementation (which is done in the source files like `.c` files). For example:

```
c
int add(int, int); // Function declaration in a header file
```

 Copy code

- Constants and Macros:** They can define constants or macros that can be used in the program.

```
c
#define PI 3.14
```

 Copy code

- Structure Definitions:** They may include structure definitions, typedefs, or other types.

```
c
typedef struct {
    int x, y;
} Point;
```

 Copy code

4. How Are Header Files Included?

- You use the `#include` directive to include header files in your `.c` files. This allows you to access the functions, constants, and other elements defined in the header file.

```
c
#include <stdio.h> // Standard C library header file
#include "myheader.h" // Custom header file in the same directory
```

 Copy code

- Standard Header Files:** The ones in angle brackets (`< >`) refer to standard library header files that are pre-installed with the compiler (e.g., `stdio.h`, `stdlib.h`).
- Custom Header Files:** The ones in double quotes (`" "`) refer to your own header files that you or others create for your project.

Example of How Header Files Work:

mathfunctions.h (Header File)

```
c
// Copy code

#ifndef MATHFUNCTIONS_H // Prevent multiple inclusion
#define MATHFUNCTIONS_H

int add(int, int);
int multiply(int, int);

#endif
```

mathfunctions.c (Source File)

```
c
// Copy code

#include "mathfunctions.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

main.c (Main Program File)

```
c
// Copy code

#include <stdio.h>
#include "mathfunctions.h" // Include custom header file

int main() {
    int result = add(3, 4);
    printf("Sum: %d\n", result);
    return 0;
}
```

Key Points About Install Software

When your coursework says "install the software", they mean:

- ✚ Take the executable file (the `.out` or compiled program) that you made from your C code,
- ✚ and move it into a system-wide location on your Minix or Linux operating system — specifically into the `/usr/bin/` folder — so that you can run your program from anywhere on the system.

In normal Linux/Unix systems:

- `/usr/bin/` is a directory that holds many command-line programs (like `ls`, `cat`, `cp`, etc.)
- When you "install" your own program into `/usr/bin/`, you are making it available like a real command (for example, typing just `encryption` in any folder will run your program).

When your coursework says "install the software", they mean:

- ✚ Take the executable file (the `.out` or compiled program) that you made from your C code,
- ✚ and move it into a system-wide location on your Minix or Linux operating system — specifically into the `/usr/bin/` folder — so that you can run your program from anywhere on the system.

In normal Linux/Unix systems:

- `/usr/bin/` is a directory that holds many command-line programs (like `ls`, `cat`, `cp`, etc.)
- When you "install" your own program into `/usr/bin/`, you are making it available like a real command (for example, typing just `encryption` in any folder will run your program).

Quick Summary

Step	What you do	Command Example
1	Compile your program	<code>gcc encryption.c -o encryption</code>
2	Become root	<code>su</code> and enter password
3	Move the compiled program to <code>/usr/bin/</code>	<code>mv encryption /usr/bin/</code>
4	Now you can run it from anywhere	<code>encryption file.txt encryptedfile.txt</code>